# Ders 10 Çalışma Özeti

## Ders 10 Çalışma Özeti

### Genel Konular

- Large Class: aşırı büyük sınıf (ekrandan, 00:10:00)
  - Sınıfın büyük olması, sınıfa ilgisiz sorumluluklar atandığının göstergesidir; böylece uyum düşer. Sorumluluklar birbiriyle ilgili olsa bile çok sayıda sorumluluk sınıfı hantal, anlaşılması zor ve karmaşık yapar.
  - Göstergeler: kod satırı (line of code) cinsinden büyüklük, küçük metotlar bile olsa metot sayısının çok olması ve üye alan sayısının fazlalığı (bu alanların erişim metotları da sınıfı şişirir).
  - Çözüm: Extract Class, Extract Subclass, Duplicate Observed Data.
- Extract Class: sınıf çıkartma (00:12:16)
  - Birbiriyle ilgisiz ya da ilgisi uzak iki iş yapan sınıf, bu işlere göre bölünür. Birlikte anlam ifade eden üye alanlar ve onları en yoğun kullanan metotlar yeni sınıfa taşınır.
  - Eski sınıftan yeni sınıfa genelde sahiplik ilişkisi kurulur. Örnek: `Person` sınıfındaki ofis alan kodu, ofis numarası ve `getTelephoneNumber` yeni `TelephoneNumber` sınıfına taşınır; `Person.getTelephoneNumber` yalnız `officeTelephone.getTelephoneNumber()` çağrısını iletecek şekilde kalır.
  - Şirketlerin de telefonu olabileceğinden bu eylem sistem genelindeki bir Duplicated Code sorununu da çözer.
- Extract Subclass: alt sınıf çıkartma (ekrandan, 00:16:00)
  - Bir sınıfın bazı üyeleri her zaman kullanılmıyorsa bu üyelerden yeni bir alt sınıf oluşturulur.
  - Örnek: `JobItem` (`getTotalPrice`, `getUnitPrice`, `getEmployee`) bölünür; belirli bir çalışana yaptırılan işler için `LaborItem` alt sınıfı oluşur (`getUnitPrice`, `getEmployee`).
  - Geriye doğru uyumluluk gerekiyorsa `JobItem.getEmployee` bırakılıp `null` döndürebilir; maliyeti düşükse tamamen kaldırılması daha doğrudur.
- Duplicate Observed Data: gözlenen verinin çoğullanması (ekrandan, 00:20:00)
  - Uygulama alanıyla ilgili bir verinin yalnız bir GUI sınıfında saklanması doğru değildir. Veri, uygulama alanı/bilgi katmanında yeni bir sınıfa taşınır; GUI nesnesi ile yeni bilgi nesnesi arasında Observer kalıbı kurulur.
  - Örnek: `IntervalWindow` hem `startField`, `endField`, `lengthField` alanlarını ve `FocusLost` olaylarını hem de `calculateLength`, `calculateEnd` iş mantığını içerir. Sonra `Interval` sınıfı (`start`, `end`, `length`, `calculateLength`, `calculateEnd`) ayrılır; `IntervalWindow` → `Interval` 1 çokluklu sahiplik kurulur. `IntervalWindow` `Observer` arayüzünü gerçekler, `Interval` `Observable`'dan türer.
  - Observer yalnız iş mantığındaki veri değişince GUI'nin kendiliğinden güncellenmesi gerekiyorsa (örneğin dağıtık sistemlerde) gereklidir; normal veri girişinde GUI zaten olayları yakalayıp iş nesnesine iletir.
- Divergent Change ve Shotgun Surgery: değişikliklerin farklı noktalarda yapılmasının gerekmesi (ekrandan, 00:24:30)
  - Gereksinim değişikliği ya da hata düzeltmesi için yapılan değişikliklerin mümkün olduğunca az yerde yapılması tercih edilir. İyi bir tasarımda bir değişiklik en fazla bir iki sınıfı ya da bir sınıfın bir iki metodunu etkiler.
  - Değişiklikler aynı sınıfın farklı metotlarında yapılıyorsa Divergent Change, birden fazla sınıfta yapılıyorsa Shotgun Surgery vardır.
  - Slayttaki örnekler: yeni bir veritabanı ile çalışmak gerekirse bir sınıfın 3 metodunun değişmesi (Divergent Change); yeni bir tür kredi söz konusu olunca 2 sınıftaki 5

metodun değişmesi (Shotgun Surgery).
  - Çözüm: Extract Class; Move Method / Move Field (üyeler daha uygun bir mevcut sınıfa taşınır).
- Move Method: metot taşınması (ekrandan, 00:30:00)
  - Bir metot tanımlandığı sınıftan çok başka sınıflardan çağrılıyorsa, en çok hangi sınıftan çağrılıyorsa o sınıfa taşınmalıdır.
  - Metot eski sınıftan tamamen silinir ya da eski sınıftaki hali yeni yerdeki metodu çağıracak şekilde bırakılır (delegation).
  - Çok sık kullanılır ve Extract Class gibi bazı eylemlerin alt adımıdır.
- Primitive Obsession: ilkel tiplerin aşırı kullanımı (ekrandan, 00:31:30)
  - Yapısal programlamadan gelenler ilkel tipleri (int, char, boolean gibi tek birimlik veri) aşırı yoğun kullanır.
  - Kusur kelimelerle tarif edilmek yerine her biçimine yönelik eylem üzerinden incelenir: Replace Data Value with Object, Replace Type Code with Class, Replace Type Code with Subclasses, Replace Type Code with State/Strategy, Replace Array with Object. Extract Class ve Introduce Parameter Object da daha nesneye yönelik bir program için kullanılabilir.
- Replace Data Value with Object (ekrandan, 00:34:30)
  - Bir üye alan, hele bu alanla ilgili çeşitli davranışlar varsa, bir sınıfla simgelenmelidir.
  - Örnek: `Order` sınıfındaki `customer: String` alanı yerine `name: String` alanlı bir `Customer` sınıfı oluşturulur. Belli bir müşterinin tüm siparişlerinin aranması gibi işlemler çoğaldıkça müşteri sınıfı ayrılmazsa sipariş sınıfı aşırı büyür.
  - Hoca bir siparişin tek müşteri tarafından verildiğini, bir müşterinin birden fazla siparişi olabileceğini söyleyerek slayttaki "1" çokluğunun yerini derste düzeltti (00:34:58-00:35:53).
- Replace Type Code with Class (ekrandan, 00:40:00)
  - Yapısal programlamadaki sabit değişken kullanma alışkanlığıyla ilgilidir.
  - Örnek: `Person` sınıfında `0`, `A`, `B`, `AB` sabitleri ve `bloodGroup: int` alanı vardır. Geçersiz bir değer (örneğin 10) atanabilir; bunu engellemek için setter içinde ayrıca kontrol gerekir.
  - Sonra `BloodGroup` sınıfı oluşturulur ve `0`, `A`, `B`, `AB` sabitleri artık `BloodGroup` türündendir; `Person` → `BloodGroup` 1 çokluklu ilişki. Bu pratikte enum kodlamaktır; kısaca enum da kullanılabilir.
- Replace Type Code with Subclasses (ekrandan, 00:44:00)
  - Bir sınıfın davranışını belirleyen ilkel bir tip kodu yerine kalıtım ve çok biçimlilik kullanmak daha uygundur.
  - Örnek: `Employee` sınıfında `ENGINEER`, `SALESMAN` sabitleri ve `type: int` alanı vardır. İlgili her metotta `switch(type)` ya da `if-else if` zinciri oluşur; yeni tür eklendikçe sınıf Large Class, metotlar Long Method olur. Kök neden tip kodudur.
  - Sonra `Employee` üst sınıf, `Engineer` ve `Salesman` alt sınıflar olur. Varsayılan durum (default/else) yoksa `Employee` soyut yapılabilir. Delegation metoduna gerek yoktur.
- Replace Type Code with State/Strategy (ekrandan, 00:50:00)
  - Kalıtım bir nedenle kullanılamıyorsa (örneğin sınıf zaten başka bir hiyerarşide yer alıyorsa ya da bazı işler bazı alt sınıflarda anlamsız kalıyorsa) State ya da Strategy kalıbı kullanılır.
  - Örnek: `Employee` → `EmployeeType` (1 çokluklu); `EmployeeType`'ın alt sınıfları `Engineer` ve `Salesman`.
  - Durumdan duruma geçiş varsa State, yalnız işin yapılış biçimi değişiyorsa Strategy seçilir.
- Replace Array with Object (ekrandan, 00:52:30)
  - Çok boyutlu dizilerde ya da birden fazla paralel dizide kayıt tutmak yerine tek bir nesne kullanılır.
  - Önce: `String[][] scores = new String[3][teamCount];` ve `scores[0][0] = "Liverpool"; scores[1][0] = "15"; scores[2][0] = "67";`. Sayısal veri string olarak tutulduğundan hesap için ayrıca dönüşüm gerekir.
  - Sonra: `Score` dizisi; `scores[0] = new Score("Liverpool"); scores[0].setPoints("15"); scores[0].setAverage("67");`.
- Feature Envy: özellik kıskançlığı (ekrandan, 01:00:00)
  - Bir metot ait olduğu sınıfın üyelerinden çok başka sınıfların üyelerine erişiyorsa bu duruma özellik kıskançlığı denir. "Çok" ortalamanın ya da %20 gibi bir eşiğin üzerin-

deki erişim sayısıdır.
  - Sorumlulukların doğru atanmadığının göstergesi olabilir. Bağlaşım ilkesi gereği hiçbir sınıf yalnız kendi metotlarını kullanmaz; öyle olsaydı uyum olumsuz etkilenirdi.
  - Strategy ve Visitor kalıpları özellik kıskançlığı olarak yorumlanmamalıdır.
  - Çözüm: Move Method; kıskançlık metodun bir kısmındaysa Extract Method ile o kısım çıkarılıp kıskanılan üyelerin sınıfına taşınır.
- Video kiralama örneğinin adım adım refactoring'i (01:19:36 sonrası)
  - Fowler'ın Refactoring kitabının 1. bölümündeki örnek Eclipse'te ve JUnit testleriyle adım adım iyileştirilir. Her adım ayrı bir paket kopyasında (`fowler_01`, `fowler_02`, …) yapılır ve her adımdan sonra testler çalıştırılır.
  - Başlangıç: `Movie (_priceCode: int)`, `Rental (_daysRented: int)`, `Customer (statement())`. En uzun ve en kusurlu sınıf `Customer`, en uzun metot `statement`, en göze batan kısım `switch` bloğudur (ekrandan, 01:20:00).
  - 1: `switch` bloğu Extract Method ile `amountFor(Rental each)` metoduna çıkarılır.
  - 2: Değişken adları anlamlı yapılır (`each` → `aRental`, `thisAmount` → `result`).
  - 3: `amountFor`, `Customer`'ın hiçbir üyesine erişmediği için Move Method ile `Rental`'a taşınır ve `getCharge` adını alır.
  - 4: `statement` içindeki `thisAmount` geçici değişkeni Replace Temp with Query ile kaldırılır.
  - 5: Bonus (frequent renter points) hesabı `Rental.getBonus()` metoduna taşınır.
  - 6: Toplamlar `Customer.getTotalCharge()` ve `getTotalBonus()` metotlarına çıkarılır.
  - 7-8: `Rental.getCharge` ve `getBonus` hesabı `Movie.getCharge(int daysRented)` ve `Movie.getBonus(int daysRented)` metotlarına taşınır.
  - 9: `Rental.getMovieTitle()` eklenir; ayrıca `addFooter()` ve `statementLine(Rental aRental)` metotları çıkarılır.
  - 10: `switch`'i tamamen kaldırmak için `Movie` soyut yapılıp `MovieNewRelease`, `MovieRegular`, `MovieChildrens` alt sınıfları oluşturulabilir (ekrandan, 02:40:20). Bu adım derste kodlanmadı.
- Kara kutu ve beyaz kutu sınaması (ekrandan, 01:50:00)
  - Kara kutu: birimin iç işleyişi bilinmez, yalnız beklenen girdilere karşı beklenen çıktıların üretilip üretilmediğine bakılır. Beyaz kutu: iç işleyiş bilinir ve sınamalar buna göre belirlenir.
  - Beyaz kutu test kalitesini artırır, ama sınırlı proje zamanını uzatabilir. Örnekte mevcut test case'lerle (kara kutu) devam edildi.
- Karşıt kalıplar (antipatterns) (02:46:25)
  - Kod kusurları küçük ölçekli kusurlardır. Daha büyük ölçekte tanımlanan kusurlara antipattern denir: iyi bir çözüm ya da tasarım kalıbı gibi görünen, ama yaklaşımın önerilerini çiğneyen kalıplar.
  - Yazılım geliştirme, yazılım mimarisi ve yazılım proje yönetimi düzeylerinde antipattern'ler tanımlanmıştır.

## Hocanın Özellikle Vurguladığı Kısımlar

- Kodun eski sınıftan tamamen silinmemesi (00:14:34)
  - Extract Class ve Move Method'da metot gövdesi yeni yere taşınır; eski sınıfta çağrıyı ileten küçük bir metot bırakılır. Buna delegation denir.
- Gizli bilgi: redundant information (00:20:42)
  - Sahiplik ilişkisi sınıf şemasında hem okla hem sınıf gövdesinde üye alan olarak gösterilebilir. Biri gösterilmişse öteki de var sayılır; kod buna göre yazılır.
- Strategy ve Visitor kalıpları Feature Envy değildir (00:59:08)
  - Visitor başka nesnelerin üzerinde işlem yapar, Strategy işi başka bir algoritma nesnesine yaptırır. Bunlar bir problemi doğru çözmek için bilinçli yapılan tasarımlardır.
- Küçük adımlarla refactoring ve sürekli test (01:25:16, 01:34:22)
  - Refactoring ezberden ya da doğaçlama yapılmaz; küçük adımlar atılır ve her adımdan sonra testler çalıştırılır. Hata yapmak kolaydır, ama hazır test case'ler hatayı hemen yakalar.
  - Derste iki kez test hatası çıktı: ilk Extract Method'da toplam satırı unutulup sonra iki kez eklendi; `getTotalBonus` kopyala-yapıştırla `double` döndürünce boş makbuz testi `0.0` yüzünden kırıldı.
- Replace Temp with Query'nin başarım bedeli (01:52:31)

  - Hoca testleri birkaç kez çalıştırıp ortalama yaklaşık 6 ms'den 10 ms'ye çıkıldığını gözledi. Bu farkın hissedilmediğini, bakım maliyetini düşürmek için eylemin yapılmasını önerdi. Eylem farklı alt sistemler arası çağrılar yüzünden çalışmayı çok uzatıyorsa geri alınabilir.
- Geriye doğru uyumluluk ile genişletilebilirlik dengesi (02:42:24)
  - 10. adım `Movie` yapıcısını ve test case'leri değiştirir; dışa açılan arayüz değiştiği için refactoring'in katı tanımı dışına çıkar. Geriye doğru uyumluluk önemliyse yapılmaz.
  - Yapılırsa yeni film türü için yalnız bir alt sınıf eklemek yeter, başka yere dokunulmaz.

## Kısa Tekrar Notları

- Large Class çözümü: Extract Class, Extract Subclass, Duplicate Observed Data.
- Duplicate Observed Data: iş verisi GUI'den bilgi katmanına taşınır; gerekiyorsa Observer kurulur (GUI = Observer, bilgi nesnesi = Observable).
- Divergent Change: tek sınıfın birçok metodu değişir. Shotgun Surgery: birçok sınıf değişir (saçma tüfeği benzetmesi). Çözüm: Extract Class, Move Method/Field.
- Move Method: metot en çok hangi sınıftan çağrılıyorsa oraya taşınır; eski yerde delegation kalabilir.
- Primitive Obsession'ın beş "Replace" eylemi: Data Value with Object, Type Code with Class, Type Code with Subclasses, Type Code with State/Strategy, Array with Object.
- Varsayılan durum yoksa tip kodunun yerine gelen üst sınıf soyut olabilir.
- Feature Envy: başka sınıfların üyelerine eşik üstü erişim; çözüm Move Method ya da Extract Method.
- Kiralama örneği 9 adımda kodlandı; 10. adım (çok biçimli `Movie` hiyerarşisi) tartışıldı.
- `Movie` arayüz değil soyut sınıf olur, çünkü `title` üye alanı ve yapıcısı vardır.
- Antipattern: tasarım kalıbının karşıtı; güzel bir çözüm gibi görünen yanlış kalıp.

## Detaylı Açıklamalar

- Large Class'ın çözüm yolları birbirini tamamlar. Extract Class'ta birbiriyle yakından ilişkili üye alan alt kümesi ve onları yoğun kullanan metotlar yeni bir sınıfa alınır; eski sınıftan yeni sınıfa sahiplik ilişkisi kurulur, böylece iletişim kesilmez. Telefon örneğinde `TelephoneNumber` sınıfına aynı alan koduna sahip numaraları bulma ya da operatöre göre en uygun hattı seçme gibi işe yarar başka metotlar da eklenebilir. Extract Subclass'ta oluşan yeni sınıf eski sınıfın alt sınıfıdır; bunun için sınıfın bazı üyelerini her durumda kullanmadığı iş mantığından yakalanmalıdır.
- Duplicate Observed Data, kullanıcı arayüzü ile iş mantığı katmanları arasındaki yüksek bağımlılıktan doğar. Özellikle veri giriş ekranlarında "bilgi hazır, hemen kullanalım" alışkanlığı, ön yüzden sorumlu sınıfın iş mantığıyla da uğraşmasına yol açar. Bu bir uyum eksikliğidir. Sonraki halde `IntervalWindow` yalnız GUI işlerini yapar; olaylardan sonra gereken işleri `Interval` üye alanına aktarıp onun metotlarını çalıştırır.
- Divergent Change ve Shotgun Surgery'de ortak nokta, tek bir değişikliğin kodda dağınık yerlere dokunmasıdır. Hoca Shotgun Surgery'yi saçma tüfeği benzetmesiyle anlattı: hedef tahtasına bir sınıf şeması asılsa tüfek çok sayıda sınıfa delik açar, tabanca ise en fazla bir sınıfın birkaç metoduna isabet eder. Bu kusurlara Türkçe ad vermedi.
- Move Method'da karar çağrı sayısına göre verilir. Hocanın örneğinde bir metot kendi sınıfından 2, X sınıfından 5, Y sınıfından 7, Z sınıfından 3 ayrı metotta çağrılıyorsa en uygun yer Y sınıfıdır.
- Replace Type Code with Class'ta `BloodGroup` sınıfının sabitleri artık `int` değil `BloodGroup` türündedir. Üye alanı ve metodu olmayan basit enum yapıları da arka planda bu mantıkla desteklenir.
- Replace Type Code with State/Strategy seçeneği şu durumda öne çıkar: bazı işler `Engineer` için anlamlıyken `Salesman` için anlamsızsa kalıtımla bölmek, Composite kalıbında yaprak düğüme çocuk ekleme metodunun boş gövdeyle kodlanması gibi sorunlar doğurur. Bu durumda iş mantığı sınıfından kalıtım yerine yalnız işin yapılış biçimi özelleştirilir.
- Replace Array with Object'te `struct` dizisi kullanmak Primitive Obsession sayılmaz; nesneye geçişte zaten bir `Score` sınıfı oluşturulur.
- Feature Envy'de "feature" üye alan anlamındadır, ama hoca kapsamı üye metotları da içe-

recek biçimde genişletti. Kıskanılan üyeler taşınınca iki sınıf arasındaki ilişki de ortadan kalkıyorsa bağlaşım düşer.

- Kiralama örneğinin ayrıntıları:
  - Başlangıçtaki `switch` bloğu (ekrandan, 01:30:00 ve 02:00:00): `REGULAR` için `2.0`, gün sayısı 2'yi aşarsa `(gün - 2) * 1.5` eklenir; `NEW_RELEASE` için `gün * 3.0`; `CHILDRENS` için `1.5`, gün sayısı 3'ü aşarsa `(gün - 3) * 1.5` eklenir. Sabitler: `CHILDRENS = 2`, `REGULAR = 0`, `NEW_RELEASE = 1` (ekrandan, 02:40:00).
  - Bonus: her kiralama 1 puan; yeni çıkan film 1 günden uzun kiralanırsa 2 puan. 5. adımda "önce bir artır, şart sağlanırsa bir daha artır" yerine "şart sağlanırsa 2 döndür, yoksa 1 döndür" biçimine çevrildi (ekrandan, 02:00:00).
  - 3. adımda `amountFor` yalnız okuma amaçlı erişim yaptığı için `Rental`'a taşınırken `private`'tan `public`'e çekildi.
  - 6. adımda `while + Enumeration` yerine for-each döngüsü kullanıldı (ekrandan, 02:09:30).
  - 7. adımda hesap iki parçaya bölündü: kiralama gün sayısı `Rental`'ın, film türü `Movie`'nin verisidir. `Rental`, `Movie`'ye gün sayısını verir; `Movie` türüne göre bedeli hesaplar.
  - 9. adımda `Customer`'dan `Movie`'ye giden tek doğrudan çağrı (`getTitle`) kaldırıldı. Sahiplik açısından dolaylı ilişki sürer, ama nesnelerin konuşması katmanlaşır: `Customer` yalnız `Rental` ile, `Rental` yalnız `Movie` ile konuşur.
  - Yorum satırları ("show figures for this rental", "add footer lines") ayrı bir alt işin işaretidir; bu kısımlar `statementLine` ve `addFooter` metotlarına çıkarılınca yoruma gerek kalmaz. Sonunda `Customer`'ın en uzun metodu 10 satır kadardır.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.